03. UDP 트래커 구현 상세 분석

개요

이 문서는 Torrust Tracker의 UDP 트래커 구현을 상세히 분석합니다. UDP 트래커는 BEP 15 프로토콜을 구현하여 BitTorrent 클라이언트와 효율적인 통신을 제공합니다. HTTP 트래커와 달리 UDP를 사용하여 더 낮은 오버헤드와 빠른 응답 시간을 제공합니다.

3.1 BEP 15 프로토콜 개요

프로토콜 특징

UDP 트래커 프로토콜은 다음과 같은 특징을 가집니다:

- **무연결 프로토콜**: TCP와 달리 연결 상태를 유지하지 않음
- **낮은 오버헤드**: HTTP 헤더가 없어 패킷 크기가 작음
- **빠른 응답**: 연결 설정 과정이 없어 즉시 통신 가능
- **IP 스푸핑 방지**: Connection ID를 통한 인증 메커니즘

지원하는 요청 타입

UDP 트래커는 세 가지 주요 요청을 처리합니다:

1. **Connect**: Connection ID 획득
2. **Announce**: 피어 상태 알림 및 피어 목록 요청
3. **Scrape**: 토렌트 통계 정보 요청

패킷 구조 기본 원칙

모든 UDP 패킷은 다음 원칙을 따릅니다:

- **네트워크 바이트 순서**: 모든 값은 빅 엔디안으로 전송
- **고정 크기 필드**: 각 필드는 명확히 정의된 크기를 가짐
- **Transaction ID**: 요청과 응답을 매칭하기 위한 식별자

3.2 Connect 요청 처리 과정

Connect 요청의 목적

Connect 요청은 IP 스푸핑을 방지하기 위한 Connection ID를 획득하는 과정입니다.

요청 처리 흐름

```
// packages/udp-tracker-server/src/handlers/connect.rs
pub  async  fn  handle_connect(
client_socket_addr: SocketAddr,
server_service_binding: ServiceBinding,
request: &ConnectRequest,
connect_service: &Arc<ConnectService>,
opt_udp_server_stats_event_sender: &crate::event::sender::Sender,
cookie_issue_time: f64,
) -> Response
```

1. 요청 검증

Connect 요청은 다음 필드를 포함합니다:

```
pub struct ConnectRequest {
pub transaction_id: TransactionId,
}
```

- **Protocol ID**: 매직 상수 `0x41727101980` (BEP 15에서 정의)
- **Action**: `0` (Connect 요청을 나타냄)
- **Transaction ID**: 클라이언트가 생성한 랜덤 값

2. Connection ID 생성

```
// packages/udp-tracker-core/src/services/connect.rs
pub async fn handle_connect(
&self,
client_socket_addr: SocketAddr,
server_service_binding: ServiceBinding,
cookie_issue_time: f64,
) -> ConnectionId {
let connection_id = make(
gen_remote_fingerprint(&client_socket_addr),
cookie_issue_time
).expect("it should be a normal value");
// 통계 이벤트 발송
if let Some(udp_stats_event_sender) = self.opt_udp_core_stats_event_sender.as_d
udp_stats_event_sender.send(Event::UdpConnect {
```

```
connection: ConnectionContext::new(client_socket_addr, server_service_binding),
}).await;
}
connection_id
}
```

3. Connection Cookie 알고리즘

Connection ID는 다음 알고리즘으로 생성됩니다:

```
// packages/udp-tracker-core/src/connection_cookie.rs
pub fn make(fingerprint: u64, issue_at: f64) -> Result<Cookie, ConnectionCooki
if !issue_at.is_normal() {
return Err(ConnectionCookieError::ValueNotNormal {
not_normal_value: issue_at,
});
}

let cookie = assemble(fingerprint, issue_at);
let cookie = encode(cookie);
Ok(zerocopy::FromBytes::read_from(cookie.as_slice()).expect("it should be the sa
}

fn assemble(fingerprint: u64, issue_at: f64) -> CookiePlainText {
```

```
let issue_at: byteorder::I64<NativeEndian> =
*zerocopy::FromBytes::ref_from(&issue_at.to_ne_bytes()).expect("it should be ali
let fingerprint: byteorder::I64<NativeEndian> =
*zerocopy::FromBytes::ref_from(&fingerprint.to_ne_bytes()).expect("it should be


let cookie = issue_at.get().wrapping_add(fingerprint.get());
let cookie: byteorder::I64<NativeEndian> =
*zerocopy::FromBytes::ref_from(&cookie.to_ne_bytes()).expect("it should be align


*CipherArrayBlowfish::from_slice(cookie.as_bytes())
}
```

알고리즘 단계:

1. 클라이언트 IP 주소에서 fingerprint 생성
2. 현재 시간(issue_at)과 fingerprint를 결합
3. Blowfish 암호화로 cookie 값 암호화
4. 암호화된 값을 Connection ID로 반환

4. 응답 생성

```
fn build_response(request: ConnectRequest, connection_id: ConnectionId) -> Resp
let response = ConnectResponse {
transaction_id: request.transaction_id,
connection_id,
};
```

```
Response::from(response)

}
```

Connect 응답 패킷 구조

| 오프셋 | 타입 | 필드명 | 설명 |

|--------|------|--------|------|

| 0 | i32 | action | 응답 타입 (0 = Connect) |

| 4 | i32 | transaction_id | 요청의 transaction_id와 동일 |

| 8 | i64 | connection_id | 생성된 Connection ID |

3.3 Announce 요청 처리 과정

Announce 요청의 목적

Announce 요청은 피어가 자신의 상태를 트래커에 알리고, 동일한 토렌트를 공유하는 다른 피어들의 목록을 받는 과정입니다.

요청 처리 흐름

```
// packages/udp-tracker-server/src/handlers/announce.rs

pub  async  fn  handle_announce(

announce_service: &Arc<AnnounceService>,

client_socket_addr: SocketAddr,

server_service_binding: ServiceBinding,

request: &AnnounceRequest,
```

```
core_config: &Arc<Core>,
opt_udp_server_stats_event_sender: &crate::event::sender::Sender,
cookie_valid_range: Range<f64>,
) -> Result<Response, (Error, TransactionId, UdpRequestKind)>
```

1. Connection ID 인증

```
// packages/udp-tracker-core/src/services/announce.rs
fn  authenticate(
remote_addr: SocketAddr,
request: &AnnounceRequest,
cookie_valid_range: Range<f64>,
) -> Result<f64, ConnectionCookieError> {
check(
&request.connection_id,
gen_remote_fingerprint(&remote_addr),
cookie_valid_range,
)
}
```

인증 과정:

1. Connection ID 복호화
2. 클라이언트 fingerprint 재생성

3. 발급 시간 추출 및 유효성 검증

4. 시간 범위 내 여부 확인

2. 화이트리스트 권한 확인

```
async fn authorize(&self, info_hash: &InfoHash) -> Result<(), WhitelistError>

self.whitelist_authorization.authorize(info_hash).await

}
```

3. 피어 정보 추출 및 처리

```
// packages/udp-protocol/src/peer_builder.rs

pub fn from_request(announce_request: &aquatic_udp_protocol::AnnounceRequest,

peer::Peer {

peer_id: announce_request.peer_id,

peer_addr: SocketAddr::new(*peer_ip, announce_request.port.0.into()),

updated: CurrentClock::now(),

uploaded: announce_request.bytes_uploaded,

downloaded: announce_request.bytes_downloaded,

left: announce_request.bytes_left,

event: announce_request.event.into(),

}

}
```

중요 사항:

- 요청의 IP 주소 필드는 무시됨
- UDP 패킷의 실제 송신자 IP를 사용
- 포트는 요청에서 제공된 값 사용

4. 트래커 코어 처리

```
let announce_data = self
.announce_handler
.handle_announcement(&info_hash, &mut peer, &remote_client_ip, &peers_wanted)
.await?;
```

5. 응답 생성

```
fn build_response(
remote_addr: SocketAddr,
request: &AnnounceRequest,
core_config: &Arc<Core>,
announce_data: &AnnounceData,
) -> Response {
if remote_addr.is_ipv4() {
// IPv4 피어 목록 생성
let announce_response = AnnounceResponse {
fixed: AnnounceResponseFixedData {
transaction_id: request.transaction_id,
```

```
announce_interval: AnnounceInterval(I32::new(i64::from(core_config.announce_poli

leechers: NumberOfPeers(I32::new(i64::from(announce_data.stats.incomplete) as i3

seeders: NumberOfPeers(I32::new(i64::from(announce_data.stats.complete) as i32))

},

peers: announce_data

.peers

.iter()

.filter_map(|peer| {

if  let IpAddr::V4(ip) = peer.peer_addr.ip() {

Some(ResponsePeer::<Ipv4AddrBytes> {

ip_address: ip.into(),

port: Port(peer.peer_addr.port().into()),

})

} else {

None

}

})

.collect(),

};

Response::from(announce_response)

} else {

// IPv6 피어 목록 생성 (유사한 로직)

}
```

}

Announce 요청 패킷 구조

| 오프셋 | 타입 | 필드명 | 설명 |

|--------|------|--------|------|

| 0 | i64 | connection_id | Connect에서 받은 Connection ID |

| 8 | i32 | action | 1 (Announce) |

| 12 | i32 | transaction_id | 클라이언트 생성 ID |

| 16 | 20바이트 | info_hash | 토렌트 식별자 |

| 36 | 20바이트 | peer_id | 피어 식별자 |

| 56 | i64 | downloaded | 다운로드한 바이트 수 |

| 64 | i64 | left | 남은 바이트 수 |

| 72 | i64 | uploaded | 업로드한 바이트 수 |

| 80 | i32 | event | 이벤트 타입 (0=none, 1=completed, 2=started, 3=stopped) |

| 84 | i32 | IP address | 피어 IP (무시됨) |

| 88 | i32 | key | 클라이언트 키 |

| 92 | i32 | num_want | 원하는 피어 수 |

| 96 | i16 | port | 피어 포트 |

Announce 응답 패킷 구조

고정 부분:

| 오프셋 | 타입 | 필드명 | 설명 |

|--------|------|--------|------|

| 0 | i32 | action | 1 (Announce) |

| 4 | i32 | transaction_id | 요청의 transaction_id |

| 8 | i32 | interval | 재공지 간격 (초) |

| 12 | i32 | leechers | 리처 수 |

| 16 | i32 | seeders | 시더 수 |

가변 부분 (IPv4 피어 목록):

각 피어당 6바이트:

- 4바이트: IPv4 주소
- 2바이트: 포트

가변 부분 (IPv6 피어 목록):

각 피어당 18바이트:

- 16바이트: IPv6 주소
- 2바이트: 포트

3.4 Scrape 요청 처리 과정

Scrape 요청의 목적

Scrape 요청은 하나 이상의 토렌트에 대한 통계 정보(시더 수, 리처 수, 완료 수)를 조회하는 과정입니다.

요청 처리 흐름

```
// packages/udp-tracker-server/src/handlers/scrape.rs

pub  async  fn  handle_scrape(
```

```
scrape_service: &Arc<ScrapeService>,

client_socket_addr: SocketAddr,

server_service_binding: ServiceBinding,

request: &ScrapeRequest,

opt_udp_server_stats_event_sender: &crate::event::sender::Sender,

cookie_valid_range: Range<f64>,

) -> Result<Response, (Error, TransactionId, UdpRequestKind)>
```

1. Connection ID 인증

Announce와 동일한 인증 과정을 거칩니다.

2. 토렌트 통계 조회

```
// packages/udp-tracker-core/src/services/scrape.rs

let  scrape_data = self

.scrape_handler

.handle_scrape(&Self::convert_from_aquatic(&request.info_hashes))

.await?;


fn  convert_from_aquatic(aquatic_infohashes: &[aquatic_udp_protocol::common::Inf

aquatic_infohashes.iter().map(|&x| x.into()).collect()

}
```

3. 응답 생성

```
fn  build_response(request: &ScrapeRequest, scrape_data: &ScrapeData) -> Respons

let  mut  torrent_stats: Vec<TorrentScrapeStatistics> = Vec::new();


for  file  in &scrape_data.files {

let  swarm_metadata = file.1;


let  scrape_entry = TorrentScrapeStatistics {

seeders: NumberOfPeers(I32::new(i64::from(swarm_metadata.complete) as i32)),

completed: NumberOfDownloads(I32::new(i64::from(swarm_metadata.downloaded) as i3

leechers: NumberOfPeers(I32::new(i64::from(swarm_metadata.incomplete) as i32)),

};


torrent_stats.push(scrape_entry);

}


let  response = ScrapeResponse {

transaction_id: request.transaction_id,

torrent_stats,

};


Response::from(response)
```

}

Scrape 요청 패킷 구조

| 오프셋 | 타입 | 필드명 | 설명 |

|--------|------|--------|------|

| 0 | i64 | connection_id | Connect에서 받은 Connection ID |

| 8 | i32 | action | 2 (Scrape) |

| 12 | i32 | transaction_id | 클라이언트 생성 ID |

| 16+20*n | 20바이트 | info_hash | 조회할 토렌트 해시 (반복) |

Scrape 응답 패킷 구조

고정 부분:

| 오프셋 | 타입 | 필드명 | 설명 |

|--------|------|--------|------|

| 0 | i32 | action | 2 (Scrape) |

| 4 | i32 | transaction_id | 요청의 transaction_id |

가변 부분:

각 토렌트당 12바이트:

| 오프셋 | 타입 | 필드명 | 설명 |

|--------|------|--------|------|

| 8+12*n | i32 | seeders | 시더 수 |

| 12+12*n | i32 | completed | 완료 다운로드 수 |

| 16+12*n | i32 | leechers | 리처 수 |

3.5 Connection ID 관리 및 보안

Connection Cookie 시스템

Connection ID는 실제로는 암호화된 쿠키로, 다음 정보를 포함합니다:

1. **클라이언트 Fingerprint**: IP 주소 기반 해시
2. **발급 시간**: 쿠키 생성 시점
3. **암호화**: Blowfish 알고리즘 사용

보안 메커니즘

1. IP 스푸핑 방지

```
pub fn gen_remote_fingerprint(remote_addr: &SocketAddr) -> u64 {
let mut state = DefaultHasher::new();
remote_addr.hash(&mut state);
state.finish()
}
```

- 클라이언트 IP 주소를 해시하여 fingerprint 생성
- Connection ID에 fingerprint 포함
- 다른 IP에서는 유효한 Connection ID 생성 불가

2. 시간 기반 만료

```
pub fn check(cookie: &Cookie, fingerprint: u64, valid_range: Range<f64>) -> Re
// ... 복호화 과정 ...
```

```
if issue_time < valid_range.start {
return Err(ConnectionCookieError::ValueExpired {
expired_value: issue_time,
min_value: valid_range.start,
});
}

if issue_time > valid_range.end {
return Err(ConnectionCookieError::ValueFromFuture {
future_value: issue_time,
max_value: valid_range.end,
});
}

Ok(issue_time)
}
```

- 일반적으로 2분 유효 기간 설정
- 만료된 Connection ID는 자동 거부
- 미래 시간의 Connection ID도 거부

3. 암호화 보안

```
fn encode(mut cookie: CookiePlainText) -> CookieCipherText {
```

```
let cipher = Current::get_cipher_blowfish();
cipher.encrypt_block(&mut cookie);
cookie
}

fn decode(mut cookie: CookieCipherText) -> CookiePlainText {
let cipher = Current::get_cipher_blowfish();
cipher.decrypt_block(&mut cookie);
cookie
}
```

- Blowfish 대칭 암호화 사용
- 서버만 알고 있는 키로 암호화
- 클라이언트는 Connection ID 내용을 알 수 없음

3.6 IP 스푸핑 방지 메커니즘과 보안 고려사항

스푸핑 공격 시나리오

1. **기본 UDP 취약점**: UDP는 무연결 프로토콜로 송신자 IP 위조 가능
2. **트래커 오남용**: 가짜 IP로 announce하여 잘못된 피어 정보 제공
3. **DDoS 증폭**: 트래커를 이용한 반사 공격

방어 메커니즘

1. Connection ID 기반 인증

```
// Connection ID 생성 시 클라이언트 IP 포함
let  connection_id = make(gen_remote_fingerprint(&client_socket_addr), cookie_is
.expect("it should be a normal value");


// 후속 요청에서 IP 검증
fn  authenticate(
remote_addr: SocketAddr,
request: &AnnounceRequest,
cookie_valid_range: Range<f64>,
) -> Result<f64, ConnectionCookieError> {
check(
&request.connection_id,
gen_remote_fingerprint(&remote_addr),
cookie_valid_range,
)
}
```

동작 원리:

- Connect 요청 시 클라이언트 IP를 Connection ID에 바인딩
- 후속 요청에서 동일한 IP에서만 유효한 Connection ID 사용 가능
- IP가 다르면 Connection ID 검증 실패

2. 실제 IP 주소 사용 강제

```
// 요청의 IP 필드 무시, 실제 송신자 IP 사용
let remote_client_ip = client_socket_addr.ip();
let mut peer = peer_builder::from_request(request, &remote_client_ip);
```

보안 효과:

- 클라이언트가 요청에 포함한 IP 주소 무시
- UDP 패킷의 실제 송신자 IP만 사용
- IP 스푸핑으로도 잘못된 피어 정보 등록 불가

3. 시간 제한 및 재사용 방지

```
// Connection ID 유효 기간 제한 (일반적으로 2분)
const COOKIE_LIFETIME: f64 = 120.0; // 초

// 발급 시간 기반 검증
if issue_time < valid_range.start {
return Err(ConnectionCookieError::ValueExpired { ... });
}
```

추가 보안 고려사항

1. Rate Limiting

실제 구현에서는 다음과 같은 제한이 필요합니다:

- IP별 요청 빈도 제한

- Connection ID 오류 횟수 제한
- 비정상적인 패턴 감지

2. 로그 및 모니터링

```
// 통계 이벤트 발송으로 모니터링 지원
if  let Some(udp_stats_event_sender) = self.opt_udp_core_stats_event_sender.as_d
udp_stats_event_sender.send(Event::UdpConnect {
connection: ConnectionContext::new(client_socket_addr, server_service_binding),
}).await;
}
```

3. 프로토콜 확장 고려

- BEP 41 (UDP Tracker Protocol Extensions) 지원 준비
- 향후 보안 강화 방안 수용 가능한 구조

결론

Torrust Tracker의 UDP 트래커 구현은 BEP 15 프로토콜을 충실히 따르면서도 강력한 보안 메커니즘을 제공합니다. Connection ID 기반의 IP 스푸핑 방지, 시간 기반 만료, 암호화된 쿠키 시스템을 통해 안전하고 효율적인 피어 중개 서비스를 제공합니다.

주요 특징:

- **효율성**: UDP 기반으로 낮은 오버헤드
- **보안성**: 다층 방어 메커니즘
- **확장성**: 모듈화된 구조로 유지보수 용이

- **표준 준수**: BEP 15 완전 구현

이러한 구현을 통해 BitTorrent 생태계에서 안정적이고 신뢰할 수 있는 트래커 서비스를 제공할 수 있습니다.